A Systematic Review of AI-Based Music Composition: Limitations and Breakthroughs

Byunghyun Ban[1,2]
[1]Gyeongkuk National University, Department of Music, [2]NaNa Lab
bhbban@nanalab.kr

ABSTRACT

AI-based music composition has evolved rapidly, with research paradigms shifting from modeling local patterns to capturing global musical coherence. This paper examines how compositional AI has progressed from modeling local patterns to forming global musical ideas. Early machine learning models, based on state transition probabilities, were able to capture only local continuity between notes and chords, and thus failed to produce meaningful phrasing or large-scale musical structure. With the introduction of recurrent architectures, models began to incorporate temporal context, and transformer-based approaches further enabled the generation of musically coherent passages over longer spans. Nevertheless, these models still exhibit limitations in maintaining overall structural completeness and long-term musical direction. More recent approaches move beyond sequential prediction toward the simultaneous generation of musical structure and sonic characteristics. This shift reflects a transition from the accumulation of local decisions to the formation of a more holistic musical representation.

▪Keyword : compositional AI, AI-based composition, generative AI, pattern reconstruction

I. Introduction

Algorithmic approaches to music composition have been studied much earlier than modern machine learning-based generative models. Over time, music generation techniques have evolved from rule-based systems to probabilistic models, and subsequently to deep learning-based approaches, progressively expanding from the generation of local patterns to the formation of global musical structure.

This study surveys this technological evolution and analyzes how different approaches model musical structure. Based on this analysis, we discuss the extent to which AI-based composition exhibits structural similarities to human creative processes.

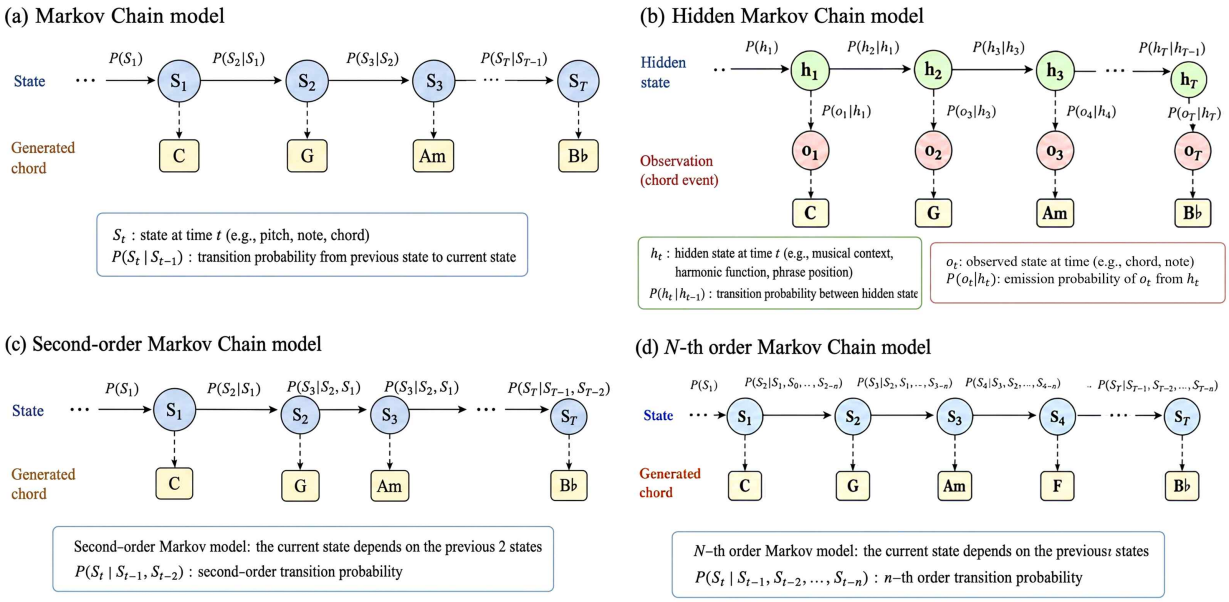
II. Non-learning Algorithms

1. Rule-based compositions

The earliest known computer-generated musical score is the Illiac Suite, composed in 1957 [1]. The Illiac Suite represents a prototypical rule-based generative system, in which compositional rules are explicitly encoded in the program. Subsequently, a wide range of studies refined and extended algorithmic and rule-based approaches to music composition. While some works employed arbitrary or ad hoc rules, academically grounded approaches typically derived compositional patterns from established music theory, such as counterpoint and harmony, and used them as generative rules [2–5].

2. Limitations

However, due to the inherently hierarchical structure of music and its complex temporal dependencies, algorithms that directly formalize theoretical rules often produce outputs that are locally valid but fail to maintain coherent phrasing and large-scale structure. In addition, inconsistencies arising from



[Fig. 1] Conceptual comparison of Markov-based models for chord generation.
Blue circles denote states, green circles denote hidden states, pink circles denote observations, and beige boxes denote generated chord outputs. Arrows indicate state transitions, and dashed arrows indicate observation or output generation. (a) A first-order Markov Chain model, in which the current state depends only on the immediately preceding state. (b) A Hidden Markov Chain model, in which latent hidden states generate observable chord events through emission probabilities. (c) A second-order Markov Chain model, in which the current state depends on the previous two states. (d) An n-th order Markov Chain model, in which the current state depends on the previous n states.

incomplete or conflicting rule sets are frequently observed [6]. Although some studies attempted to address these limitations by introducing empirically designed rules [7–8], such approaches were unable to overcome the fundamental constraints of rule-based systems.

Alternative methods, including evolutionary algorithms, were later proposed [9–11]. Nevertheless, these approaches remained limited in terms of model flexibility, representational capacity, and their ability to capture long-term temporal dependencies. As a result, data-driven machine learning approaches gradually emerged as the dominant paradigm in algorithmic composition.

III. Early Machine Learning Approaches

1. The Need for Stochastic Models

Despite their grounding in established music theory, rule-based approaches are inherently limited in their ability to model the variability and uncertainty present in musical composition. In practice, music is not strictly governed by deterministic rules; rather, it exhibits probabilistic patterns, in which multiple valid continuations may exist for a given musical context. Moreover, constructing comprehensive rule sets that adequately capture diverse musical styles proved to be highly labor-intensive.

Furthermore, musical structure is inherently sequential and context-dependent, requiring models capable of representing transitions between states over time. Deterministic rule systems struggle to capture such temporal dependencies in a flexible manner, particularly when multiple competing constraints are involved.

These limitations motivated the development of stochastic approaches, in which music is modeled as a probabilistic process. Within this framework, compositional decisions are not determined by fixed rules, but are instead governed by probability distributions over possible musical events, enabling more flexible and stylistically coherent generation.

2. Early Stochastic Models

Early stochastic approaches to music composition were not yet fully developed generative models based on Bayesian learning, but were closer to frameworks that modeled transitions between musical elements. In 1955, Caplin and Prinz [12] implemented a melody generation system inspired by probabilistic transitions, similar in spirit to Mozart's dice

game. In the early 1960s, Iannis Xenakis further explored stochastic approaches to composition, proposing models based on probabilistic transitions implemented with computational assistance [13].

However, at that time, effective algorithms for learning transition probabilities directly from data had not yet been established, and computational resources were severely limited. As a result, transition probabilities were typically specified manually rather than learned from data. Consequently, these early systems can be understood as rule-driven stochastic models, combining deterministic design with probabilistic elements.

3. Markov Chain Models

3.1 Markov Chain

Markov chain is a stochastic process that evolves through a finite set of states in discrete time, assuming that the probability of the next state depends only on the current state. In algorithmic composition, this assumption allows musical generation to be formulated as a sequence of probabilistic state transitions, where musical events such as pitches, notes, or chords are encoded as discrete states.

Many early Markov-based systems modeled only a single musical viewpoint, such as pitch, rhythm, or harmony. Accordingly, the transition probability matrix was estimated from a training corpus for one selected dimension, enabling the system to reproduce local statistical regularities while remaining limited in its ability to represent interactions among multiple musical parameters.

3.2 Applications

Pinkerton (1956) calculated transition probabilities between notes based on several dozen nursery tunes and implemented the Banal Tune-Maker using these probabilities [14]. Tipei (1975) introduced Markov chains as a component within a larger compositional system [15], while Ames and Domino (1992) employed Markov chains for rhythm generation in jazz and rock contexts [16]. These examples show that Markov chains can be applied not only to melody generation, but also to rhythm generation or harmonic progression, depending on whether the inferred states are defined as notes, chords, or rhythmic events.

Attempts to compensate for the structural limitations of Markov chain models developed in two major directions. The first involved combining Markov-based generation with external structural constraints. Ponsford et al. (1999) proposed an algorithm that applied a rule-based template to Markov-generated output, thereby producing music that was more structurally acceptable [17]. The second direction involved applying Markov chains to musical domains in which local state transitions are particularly effective, such as jazz improvisation. Grachten (2001) proposed a Markov chain model for jazz improvisation [18], and Thom (2000) similarly developed an improvisation algorithm based on Markovian modeling of pitch, interval, and contour statistics [19].

3.3 Limitations

Despite their early popularity, standard Markov chains exhibit fundamental structural limitations. Because these models rely strictly on single-state transitions, their outputs may be mathematically plausible within highly localized contexts, such as valid note-to-note or chord-to-chord transitions derived from corpus statistics. However, they struggle to learn or maintain overarching musical context.

As a result, the generated output often sounds less like coherent music and more like an aimless sequence of states that merely follows short-term statistical regularities. To overcome these limitations, researchers sought models with greater flexibility and representational capacity, leading to the application of Hidden Markov Models to algorithmic composition.

4. Hidden Markov Models

4.1 Hidden Markov Model

Hidden Markov Models (HMMs) extend standard Markov chains by introducing latent, unobservable states. In a standard Markov chain, the states themselves are directly observed. In an HMM, however, the model assumes that the observed sequence is generated by an underlying sequence of hidden states.

The hidden state refers to an internal condition that cannot be directly observed but is assumed to govern the generation process. The observable state refers to the actual event that appears in the data, such as a note, chord, or rhythmic pattern. The relationship between hidden and observable states is defined by emission probabilities, which specify how likely each hidden state is to generate a particular observable event.

In musical applications, for example, a hidden state may represent an underlying chord or harmonic function, while the observable state may represent a surface-level pitch. In this

case, the emission probabilities describe which pitches are likely to be generated under a given chord. Thus, if the hidden state corresponds to a C major chord, pitches such as C, E, or G would be expected to have higher emission probabilities than non-chord tones. In this way, HMMs can model the relationship between latent harmonic context and observable melodic events.

Thus, an HMM consists of two probabilistic components: transition probabilities between hidden states, and emission probabilities from hidden states to observable outputs. HMMs have been widely used in domains where observable data reflect hidden processes, such as speech recognition, part-of-speech tagging, and genomic sequence analysis. In algorithmic composition, this structure is useful because musical surfaces, such as notes or chords, may reflect underlying harmonic, contrapuntal, or phrase-level contexts. Compared with standard Markov chains, HMMs therefore provide a more flexible framework for tasks such as harmonization, counterpoint generation, and accompaniment generation.

4.2 Applications

Biyikoglu (2003) applied HMMs to four-part chorale harmonization in the style of Johann Sebastian Bach, taking advantage of the model's ability to infer globally optimized sequences of hidden states [20]. Allan (2002) similarly employed HMMs for Bach-style chorale harmonization, but reduced the complexity of the task by decomposing the harmonization problem into three subtasks [21].

A later commercial application can be found in Microsoft's SongSmith. Morris et al. (2008) trained an HMM on approximately 300 lead sheets to generate chordal accompaniment for user-provided vocal melodies [22]. The system was designed for non-expert users, enabling them to sing or input a melody and receive a real-time harmonic accompaniment generated by the model.

These applications show that HMMs were particularly useful for tasks in which an observable musical sequence, such as a melody, must be mapped onto an underlying harmonic or contrapuntal structure. In this sense, HMMs were more suitable for harmonization and accompaniment generation than for fully autonomous composition.

4.3 Limitations

Despite their greater flexibility, HMMs retain a fundamentally Markovian temporal structure: each hidden state typically depends only on the immediately preceding hidden state. As a result, they can enrich local generation by introducing latent musical contexts, but they do not inherently solve the problem of long-range musical coherence.

In composition, this means that HMMs may generate more varied and context-sensitive outputs than simple Markov chains, yet still struggle to represent large-scale form, thematic development, or phrase-level planning. This limitation motivated later approaches, such as higher-order Markov models, which attempted to extend the temporal context by conditioning generation on multiple preceding states.

5. Higher-order Markov Models

5.1 Second order Markov Models

Second-order Markov models extend first-order Markov chains by conditioning the next state on the two immediately preceding states rather than only one. Formally, the probability of the current state is modeled as $P(s_t|s_{t-1}, s_{t-2})$, allowing the model to capture short contextual patterns that cannot be represented by single-state transitions.

In algorithmic composition, this enables the model to learn more stable local phrases, since a note, chord, or rhythmic event can be generated with reference to a short preceding context. For example, in chord progression modeling, the next chord may depend not only on the current chord but also on the previous harmonic movement.

Second-order Markov models can also be combined with hidden-state architectures. In a second-order Hidden Markov Model, the hidden state transition depends on the previous two hidden states, while observable musical events are still generated through emission probabilities. This allows the model to represent both latent musical context and short-range sequential dependency.

Farbood and Schoner (2001) proposed a second-order Hidden Markov Model for generating Palestrina-style first-species counterpoint [23]. The model was trained using rules derived from counterpoint pedagogy, allowing the system to combine hidden-state modeling with a broader short-term temporal context.

5.2 Higher-order Markov Models

Higher-order Markov models generalize this idea by conditioning the next state on the previous n states. In an n-th

order Markov model, the transition probability is defined as $P(s_t|s_{t-1}, s_{t-2}, \ldots, s_{t-n})$. By expanding the length of the conditioning context, these models aim to capture longer musical dependencies and more coherent phrase-level patterns.

In music composition, higher-order Markov models can better preserve recurring melodic, harmonic, or rhythmic sequences than first-order models. They are particularly useful when short motifs or phrase fragments need to be maintained over several consecutive events.

However, a major limitation is that the order n is fixed in advance. If n is too small, the model remains unable to capture meaningful musical context. If n is too large, the number of possible state combinations increases rapidly, causing data sparsity and computational inefficiency. As a result, higher-order models often face a trade-off between contextual depth and statistical reliability.

Ponsford et al. (1999) used third- and fourth-order Markov chains to generate Baroque sarabande-style pieces [17]. Their approach attempted to improve structural plausibility by combining higher-order transition modeling with additional structural annotations and post-processing constraints.

5.3 Variable-order Markov Models

Variable-order Markov models, also called mixed-order Markov models, were proposed to overcome the rigidity of fixed-order models. Instead of using a single predetermined context length, these models dynamically adjust the order depending on the available musical context and the statistical reliability of observed patterns.

In this framework, frequently observed patterns can be modeled with longer contexts, while rare or uncertain patterns may fall back to shorter contexts. This allows the model to use detailed contextual information when sufficient data are available, without being forced to estimate unreliable high-dimensional transition probabilities for every possible state sequence.

For algorithmic composition, variable-order models provide a more flexible way to balance local pattern preservation and generalization. They can reproduce longer motifs or stylistic patterns when those patterns are well represented in the training corpus, while still maintaining generative flexibility in less familiar contexts.

Variable-order Markov models subsequently became an important methodology in algorithmic composition from the mid-1990s onward. Conklin and Witten (1995) implemented a variable-order Markov model and proposed a multiple viewpoint system, in which different musical features, such as pitch, duration, and contour, were modeled as separate viewpoints and combined in parallel [24]. Rather than encoding all musical information into a single state sequence, this approach modeled multiple structural dimensions of music independently and then integrated them during prediction.

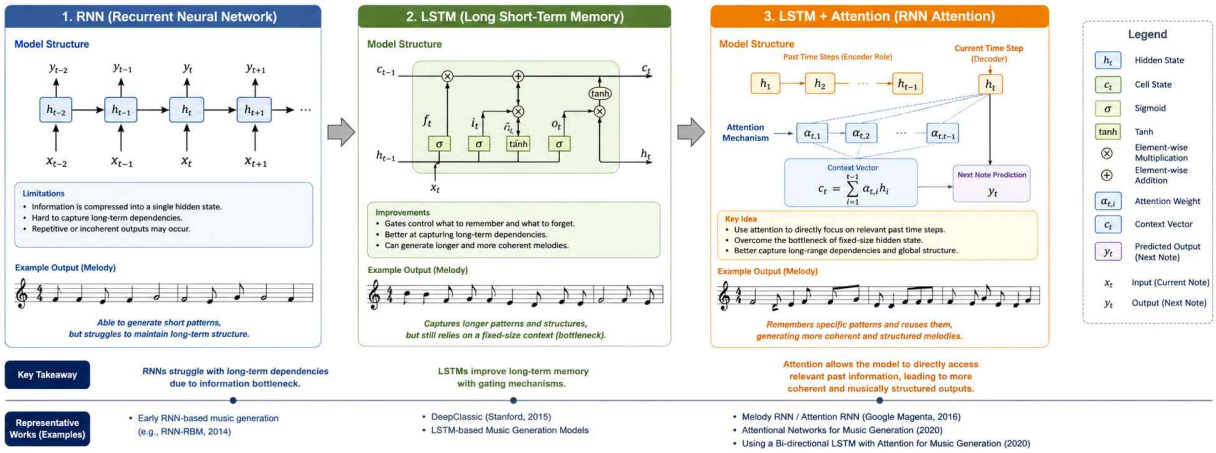
Various applications of variable-order models were later proposed [25–28]. These studies continued to explore modeling strategies in which different musical layers, such as chord, rhythm, and pitch, were represented separately and subsequently combined. At the same time, researchers also sought to reduce computational and memory complexity for real-time generation and improvisation. Representative examples include Prediction Suffix Trees, which combine variable-order Markov modeling with predictive tree structures [29], and Factor Oracles, which provide a more space-efficient representation for sequence modeling [30].

6. Limitations

Although higher-order and variable-order Markov models improved the ability to model short- and mid-range musical context, they did not fully solve the problem of global musical coherence. Increasing the order of the model substantially raises computational complexity and expands the number of possible state combinations. This often leads to data sparsity, especially when the training corpus is limited.

A related problem is weak generalization over rare transitions. Frequently occurring progressions or melodic patterns may be learned effectively, but less common yet musically important events—such as modulations, cadential gestures, or extended dominant structures—are less reliably modeled. As a result, the generated music may overuse statistically common patterns and avoid structurally important but infrequent transitions, leading to outputs that sound repetitive or stylistically flat.

Furthermore, even when longer contexts are considered, the model still predicts the next event based on preceding states. It does not inherently plan large-scale musical form, thematic development, or narrative structure. In other words, higher-order Markov models can extend local continuity, but they do not naturally generate a sense of beginning, development, climax, and resolution.



[Fig. 2] Evolution of Neural Network based Music Generation Before Transformer Era.
Standard RNNs encode previous musical events into a single hidden state, whereas LSTMs improve long-term memory through gated cell states. Attention-augmented recurrent models further allow the prediction step to selectively access relevant past states, improving the preservation of motifs, repeated patterns, and longer musical context.

These models also face a trade-off between underfitting and overfitting. If the model order or complexity is too low, the generated sequence may drift aimlessly due to insufficient contextual information. If the model order is too high, the system may become trapped in memorized patterns from the training corpus or fail to generalize to unseen transitions. Thus, while higher-order Markov models represent an important step toward context-aware music generation, they still lack a deeper understanding of musical structure.

These limitations motivated the development of models with greater representational capacity and more flexible contextual modeling, paving the way for neural network-based approaches to algorithmic composition.

IV. Artificial Neural Networks

1. Feedforward Model

Feedforward neural networks represent one of the earliest and simplest ways of applying artificial neural networks to algorithmic composition. From an application-oriented perspective, their main role was to replace explicitly defined transition tables or rule sets with learned input-output mappings. A musical context is encoded as a fixed-size input vector, and the network produces one or more musical outputs, such as a pitch, chord, voice-leading decision, or harmonization label.

Unlike Markovian models, which explicitly estimate transition probabilities between symbolic states, feedforward networks learn nonlinear relationships between musical inputs and outputs through weighted connections. This made them useful for localized prediction and harmonization tasks, especially when the target output could be formulated as a bounded classification or regression problem.

1.1 Single State Prediction

In single-state prediction, the network receives a fixed representation of the preceding musical context and predicts one target event, such as the next note, chord, or harmonic label. This formulation is conceptually similar to next-state prediction in Markov models, but the relationship between input and output is learned through the network rather than stored as an explicit transition matrix.

However, because feedforward networks lack internal memory, the temporal context must be manually encoded into the input vector. As a result, the model can only use the information explicitly provided within this fixed window, limiting its ability to represent longer musical dependencies.

Shibata (1991) proposed a harmonization model that predicted chord values or note-scale associations for a given melody using a feedforward artificial neural network [31]. Verbeurgt et al. (2004) later combined a Markov chain algorithm with an artificial neural network [32]. In their approach, the Markov chain first generated a sequence of musical motifs, and the neural network then assigned concrete

absolute pitches to these motifs one at a time.

These examples illustrate the role of feedforward neural networks in localized musical prediction. Compared with explicit Markov transition tables, feedforward networks provide a more flexible nonlinear mapping between input features and output states. However, because they lack recurrent memory, temporal context must be manually encoded into the input representation. As a result, feedforward models remained limited in their ability to infer musical context across time, which led to a rapid shift toward recurrent neural network-based approaches.

1.2 Multi-state Prediction

Feedforward architectures can also be extended to multi-state prediction, in which the network produces several musical outputs simultaneously. This approach is useful for tasks such as harmonization, counterpoint, or multi-voice generation, where the system must determine multiple notes or voices at a given moment.

Compared with single-state prediction, multi-state prediction allows the model to generate richer local musical structures. Nevertheless, the model still operates within a fixed input-output frame and does not dynamically accumulate temporal context across a sequence. Therefore, feedforward models remain most suitable for local prediction, harmonization, or refinement tasks rather than full-scale sequential composition.

HARMONET [33] was designed to infer harmonic information for four-part chorale harmonization in the style of J. S. Bach, rather than predicting isolated pitch events. Although it was not an end-to-end model, it represents an early attempt to coordinate multiple types of musical information within a neural architecture. The system consisted of three components: the first ANN extracted harmonization-related information, a constraint-based system generated chords from this information, and a second ANN added ornaments to the generated harmonic structure.

These approaches show that neural composition systems were not limited to single-state prediction. Some models attempted to infer multiple musical states or structural features in parallel, while others extended prediction along the temporal axis by generating multiple state transitions as a sequence. A patent-based implementation illustrates the latter direction. Ban (2017) proposed a modular neural composition method in which separate neural networks generate melody, rhythm, and chord sequences from seed inputs, and a post-processing network integrates these outputs into a final musical result [34]. Unlike single-state prediction models, this approach treats musical generation as the production of structured sequences, such as melody lines, rhythm patterns, and chord progressions, rather than the prediction of only one immediate next event.

2. Recurrent Model

2.1 Benefits of Applying NLP Algorithms for Music Composition

Recurrent models became attractive for algorithmic composition because music, like natural language, can be represented as an ordered sequence of symbolic events. In natural language processing, words or tokens form temporally ordered sequences governed by syntactic and semantic relationships. Similarly, in symbolic music generation, notes, chords, durations, dynamics, and rhythmic events can be encoded as tokens arranged over time.

This structural similarity allowed sequence modeling techniques originally developed for language processing to be applied to music composition. In this framework, composition can be formulated as a prediction problem: given a preceding sequence of musical events, the model predicts the next event or generates a continuation. This approach differs from feedforward models, which require a fixed-size input window, because recurrent architectures update an internal hidden state as the sequence unfolds.

From an application-oriented perspective, recurrent models provided a more natural framework for melody generation, accompaniment generation, and style imitation. Rather than predicting isolated musical events, they could process temporal sequences and preserve information from previous events through recurrent connections. This made them particularly suitable for tasks in which local continuity and short-term musical context are important.

2.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are neural architectures designed to process sequential data by maintaining an internal hidden state. At each time step, the model receives an input event and updates its hidden state based on both the current input and the previous hidden state. The updated hidden state is

then used to predict the next output. In this way, RNNs can model temporal dependencies without requiring all preceding information to be explicitly encoded in a fixed input vector.

In algorithmic composition, RNNs enabled a shift from fixed-window prediction to sequential generation. A musical sequence could be processed one event at a time, allowing the model to generate melodies, rhythms, or improvisational phrases by repeatedly predicting the next musical event. Early applications of recurrent neural networks demonstrated that neural models could learn stylistic tendencies from musical corpora and generate outputs that were more temporally continuous than those produced by simple feedforward models.

However, standard RNNs have important limitations. Because the hidden state is continuously overwritten as new inputs arrive, information from distant past events tends to be weakened or lost. This makes it difficult for RNNs to preserve long-term musical context, such as phrase-level structure, thematic recurrence, or harmonic direction over an extended passage. As a result, RNN-based music generation can produce locally coherent melodic fragments, but often struggles to maintain larger-scale musical organization.

Todd (1989) applied a three-layer recurrent neural network to the generation of monophonic melodic sequences [35]. In the same year, Duff (1989) proposed a two-layer recurrent model for generating Bach-style music. These early RNN-based approaches attempted to overcome the limitations of Markov models by learning state transitions with temporal context. However, due to computational limitations and the difficulty of training recurrent architectures at the time, these systems remained relatively small, typically employing shallow networks with only a few layers.

Franklin (2001) later proposed an RNN-based model for jazz improvisation, in which the system received a four-bar motif from a human performer and generated a four-bar improvised response [36]. The model was first trained on pre-specified examples and subsequently refined through reinforcement learning guided by heuristic rules. This paradigm, in which human-provided musical material is used as a seed for machine-generated continuation, influenced later approaches to neural music generation.

Within this broader trajectory, Ban (2017) proposed a modular sequence-level neural generation model [37], and Ban, Park, and Jung (2017) proposed an RNN autoencoder-based multi-step chord generation model [38]. These approaches extended the idea of seed-based generation by treating musical output as structured sequences, such as melody lines, rhythm patterns, and chord progressions, rather than as isolated next-state predictions.

2.3 Long-Short Term Memory

Long Short-Term Memory (LSTM) networks were introduced to address the short memory problem of standard RNNs. Unlike simple recurrent networks, LSTMs use gated memory cells that regulate the flow of information over time. These gates determine what information should be stored, updated, or forgotten, allowing the model to retain relevant information across longer sequences.

In algorithmic composition, LSTMs improved the ability of recurrent models to generate longer and more coherent musical passages. By maintaining information over extended time spans, LSTM-based systems became better suited for modeling melodic continuity, harmonic progression, and stylistic consistency. This made them useful for tasks such as melody generation, folk tune generation, accompaniment generation, and style imitation.

Nevertheless, LSTMs do not fully solve the problem of global musical structure. Their generation process remains sequential and autoregressive: each new event is generated based on previous events, without an explicit representation of the entire composition. As a result, LSTM-based systems may produce convincing short- or mid-length passages, but they often struggle to plan large-scale form, thematic development, or long-term dramatic progression. In this sense, LSTMs extend the effective memory of recurrent models, but they do not fundamentally transform sequential prediction into holistic musical planning.

In 2002, Eck and Schmidhuber proposed an LSTM-based approach to blues music generation [39]. Their model was trained to learn the long-term structure of blues music and successfully generated blues-style melodies. They argued that previous RNN-based approaches tended to focus on local representations and therefore lacked global musical structure, whereas the long-term memory mechanism of LSTMs enabled the model to capture such structure to some extent.

Subsequent studies attempted to improve performance by increasing model capacity and architectural complexity. In 2016, Sturm et al. trained LSTM models on folk music data and successfully generated new transcriptions, demonstrating

the usefulness of LSTM-based systems as tools for compositional ideation [40]. In the same year, Colombo et al. also proposed an LSTM-based composition model to address the problem of long-range temporal dependency in melody generation [41].

Oore et al. (2017) extended recurrent music generation from symbolic score generation toward expressive piano performance generation [45]. Their model predicted not only the notes to be played, but also expressive timing and dynamics, thereby treating music generation as the production of performance-like sequences rather than grid-aligned symbolic scores. Although the generated samples were reported to sound close to human piano performance, the authors also noted that the model still lacked high-level musical structure.

The introduction of LSTM networks demonstrated that neural composition models could learn both local musical patterns and longer contextual flows within a single sequential framework. This significantly influenced later developments in AI-based music composition, particularly models designed to capture longer-range musical dependencies.

2.4 Limitations

Recurrent models marked an important step beyond feedforward architectures by allowing musical sequences to be processed over time. However, their reliance on sequential prediction also introduced several limitations. First, the generation process remains inherently local: even when previous events are stored in a hidden state, the model predicts the next event step by step rather than designing the overall structure of the piece.

Second, recurrent models are sensitive to the accumulation of errors. A locally inappropriate prediction can influence subsequent predictions, causing the generated sequence to gradually drift away from the intended musical style or structure. This problem becomes more severe as the generated passage becomes longer.

Third, although LSTMs can retain longer contextual information than standard RNNs, they still do not explicitly model hierarchical musical form. Musical structure often involves relationships between motives, phrases, sections, and large-scale formal functions. These relationships are difficult to capture through a single sequential hidden state alone.

Therefore, recurrent models improved the continuity and stylistic plausibility of AI-generated music, but they remained limited in their ability to represent global form and long-range structural planning. These limitations motivated the development of attention-based architectures, especially Transformer models, which allow the model to directly access multiple positions in a sequence rather than relying solely on compressed recurrent memory.

V. From Recurrent Memory to Attention

1. Attention mechanism

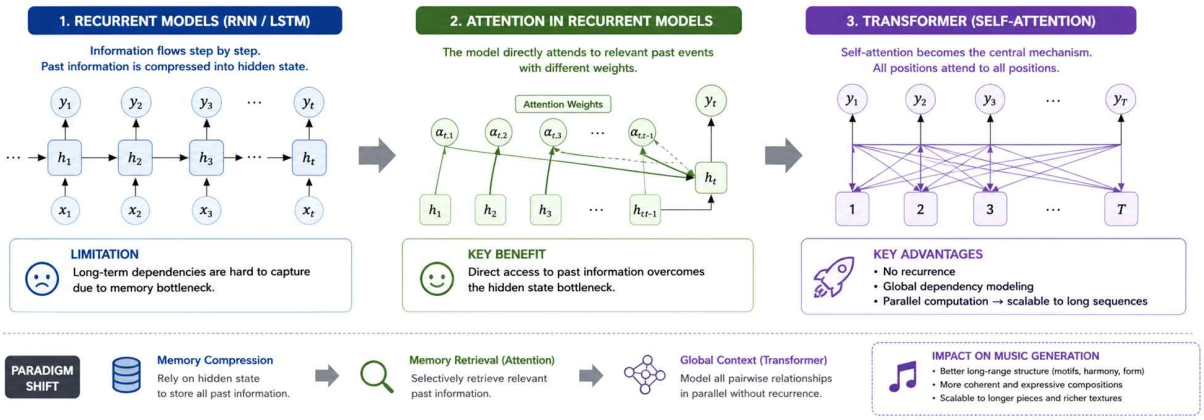
Attention mechanisms [42] were introduced to address a key limitation of recurrent architectures: the need to compress past information into a single hidden state. In RNNs and LSTMs, information from earlier time steps must be carried forward sequentially, which makes it difficult to preserve distant contextual information over long sequences. Attention provides an alternative mechanism by allowing the model to directly compare the current position with multiple previous positions and assign different weights to them according to relevance.

In music composition, this mechanism is particularly important because musical coherence often depends on non-adjacent relationships. A motif introduced in an earlier phrase may return later in varied form, a harmonic pattern may reappear after an intervening section, and rhythmic figures may structure larger formal units. By selectively attending to relevant past events, attention enables the model to capture long-range dependencies more effectively than recurrent memory alone.

Before the widespread adoption of full Transformer architectures, attention was often used as an auxiliary mechanism in recurrent music generation models. In such systems, an RNN or LSTM generated musical events sequentially, while the attention module helped the model retrieve relevant information from previous parts of the sequence. This improved the ability of recurrent models to preserve motifs, repeated patterns, and accompaniment relationships over longer contexts.

2. Transformer

The Transformer [43] architecture extended this idea by making self-attention the central mechanism for sequence modeling. Unlike recurrent models, which process sequences step by step, Transformers compute relationships among multiple positions in a sequence through self-attention. This



[Fig. 3] From recurrent memory to self-attention in neural music generation.
RNN and LSTM models encode previous musical events into compressed hidden states, whereas attention mechanisms allow the model to retrieve relevant past information directly. Transformer architectures generalize this idea through self-attention, enabling global context modeling across the entire sequence and improving long-range musical coherence.

allows the model to directly access distant events without passing information through a chain of recurrent hidden states.

This property is highly relevant to music generation. Musical structure relies heavily on repetition, variation, and self-reference across multiple temporal scales, from motives and phrases to sectional returns. Transformer-based models can therefore represent relationships between distant musical events more directly than RNNs or LSTMs. In addition, because self-attention supports parallel computation during training, Transformers can be scaled more effectively than recurrent architectures.

3. Applications

A representative example is Google Magenta's Melody RNN and Attention RNN, where attention was used to improve recurrent melody generation by enabling the model to refer to earlier musical contexts [45]. Even after the introduction of the Transformer architecture, attention-augmented LSTM models continued to be explored in music composition research. For example, a jazz composition model combining bidirectional LSTM with attention was proposed in 2022 [46].

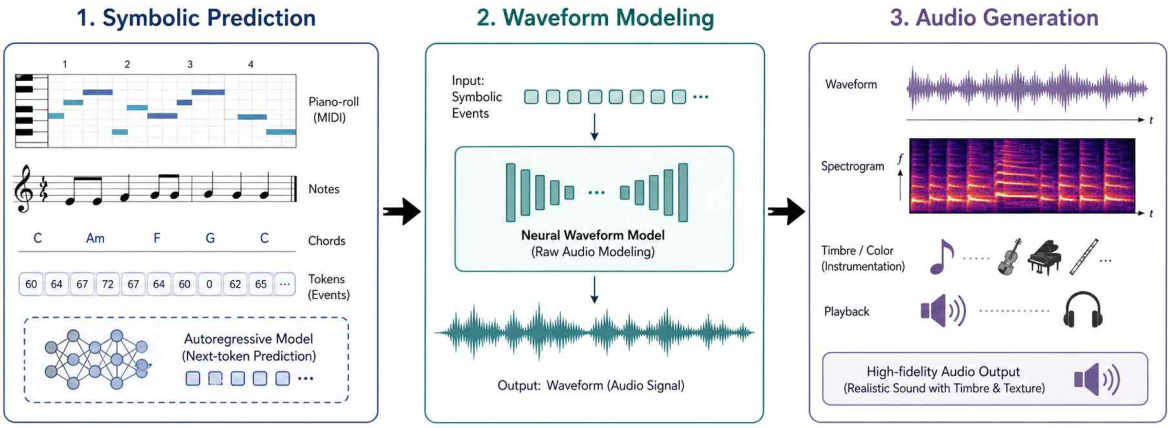
Music Transformer [47] is one of the most representative studies applying Transformer architectures to symbolic music generation. Motivated by the repetitive and self-referential nature of music across multiple temporal scales, such as motifs, phrases, and larger formal structures, the model introduced a memory-efficient relative self-attention mechanism for long musical sequences. It successfully generated minute-long piano compositions, motif continuations, and melody-conditioned accompaniments, demonstrating the potential of Transformers for long-context music generation.

OpenAI's MuseNet (2019) is another influential example of Transformer-based music generation [48]. Using a GPT-2-style architecture trained on hundreds of thousands of MIDI files, MuseNet generated music through next-token prediction, producing compositions up to four minutes long with up to ten instruments and multiple styles. This demonstrated that large-scale Transformer models could handle longer symbolic musical sequences and multi-instrumental textures.

Subsequent studies further refined Transformer-based music generation by improving musical representation and attention mechanisms. Pop Music Transformer (2020), for example, introduced a revised event representation designed to encode metrical information such as beats and bars, enabling the generation of more rhythmically coherent and expressive pop piano compositions [49]. Yu et al. later proposed Museformer, focusing on the inefficiency of full attention for long musical sequences and the difficulty of generating repetition structures. By combining fine-grained and coarse-grained attention, Museformer aimed to better capture repetition and long-range structure in symbolic music generation [50].

4. Limitations

Despite their success in long-context symbolic music generation, attention- and Transformer-based models still inherit several limitations of symbolic prediction. Most



[Fig. 4] Transition from symbolic prediction to audio-level generation in AI-based music composition.
Early symbolic models generate abstract musical events, such as notes, chords, MIDI piano-rolls, or event tokens. Waveform modeling shifts the target from symbolic structure to audio signals, allowing the model to represent sound directly. Later audio-generation approaches further incorporate waveform, spectrogram, timbre, instrumentation, and playback-level information, moving AI-based composition from score-like prediction toward the generation of music as sound itself.

Transformer-based music systems generate symbolic event sequences, such as MIDI notes, durations, velocities, instruments, chords, or metrical tokens. While such representations are effective for modeling pitch, rhythm, and formal relationships, they do not directly represent the acoustic properties of music, such as timbre, spectral texture, spatial impression, or the physical characteristics of sound.

Moreover, many Transformer-based music models remain fundamentally autoregressive. Although self-attention allows the model to access distant previous events, the generation process is still typically formulated as next-token prediction. Therefore, the model can preserve longer contextual dependencies, but it does not necessarily plan the global structure of an entire composition in advance.

This limitation is particularly important in music, where coherence is not only a matter of local continuity but also of large-scale formal organization. Motif recurrence, harmonic direction, modulation, climax, and resolution often require structural planning across an entire piece. Transformer models can learn statistical regularities associated with these phenomena, but they do not explicitly guarantee a coherent beginning, development, climax, and conclusion.

Another practical limitation is computational cost. Self-attention becomes increasingly expensive as sequence length grows, and symbolic music often produces very long token sequences, especially when multiple instruments, expressive timing, and performance details are encoded. As a result, many Transformer-based models require architectural modifications, compressed representations, or attention approximations to handle long musical sequences efficiently.

These limitations suggest that Transformer-based models substantially improved long-context dependency modeling, but they still treated music primarily as a sequence of symbolic events. The next major shift occurred when music generation began to move beyond symbolic prediction toward audio-level or spectrogram-level modeling, where structure, timbre, and texture could be generated more directly.

In other words, Transformer-based models improved the memory of symbolic composition, but they did not yet solve the problem of generating music as sound itself.

VI. Toward Waveform Generation

1. Limitations of Symbolic Prediction

Although attention- and Transformer-based models substantially improved long-context symbolic music generation, they still operate primarily on symbolic representations. In these models, music is typically represented as a sequence of discrete events, such as notes, chords, durations, velocities, instruments, or metrical tokens. This representation is effective for modeling musical structure, but it does not directly encode the acoustic substance of music itself.

The limitation is that symbolic prediction treats music as an abstract score-like sequence rather than as sound. Even when a

model successfully generates coherent pitch and rhythm patterns, the resulting output still requires a separate synthesis or rendering process to become audible music. As a result, important musical qualities such as timbre, articulation, spectral texture, spatial impression, vocal quality, and recording-like sound characteristics remain outside the direct scope of symbolic generation.

This distinction is crucial because human musical experience is not limited to symbolic structure. A listener does not perceive only pitch classes, note durations, or chord labels; rather, music is experienced as an integrated auditory phenomenon. The same melody can produce entirely different impressions depending on instrumentation, performance nuance, tone color, reverberation, and sound production. Symbolic models can describe what notes should occur, but they do not fully determine how those notes should sound.

Moreover, symbolic representations often separate composition, performance, and sound synthesis into different stages. A model may generate a MIDI sequence, but the expressive timing, dynamics, timbre, and audio rendering must be handled by additional systems or human intervention. This separation limits the ability of the model to generate music as a unified artistic object.

These limitations motivated a shift from symbolic event prediction toward audio-level generation. Instead of predicting only musical symbols, later models attempted to generate waveform, spectrogram, or latent audio representations directly. This transition allowed AI-based music generation to move closer to the generation of sound itself, incorporating not only musical structure but also timbre, texture, and acoustic detail.

2. Raw Waveform Modeling

Raw waveform modeling marked an important transition from symbolic music generation to audio-level generation. Unlike symbolic models, which generate abstract musical events such as notes, chords, or MIDI tokens, waveform models attempt to generate the audio signal itself. In this framework, music is represented as a sequence of amplitude values sampled at very high temporal resolution, often tens of thousands of samples per second. This allows the model to directly capture acoustic properties such as timbre, articulation, texture, and sound quality.

A representative milestone is WaveNet, proposed by van den Oord et al. (2016) [51]. WaveNet is a fully probabilistic autoregressive model that generates raw audio waveforms sample by sample, conditioning each sample on previous samples. Although the model was originally introduced in the context of speech synthesis, the authors also demonstrated that the same architecture could be applied to music audio generation, producing novel and realistic musical fragments. This showed that deep generative models could move beyond symbolic representations and directly model the physical waveform of sound.

Another important approach is SampleRNN, proposed by Mehri et al. (2016) [52]. SampleRNN also generates audio one sample at a time, but uses a hierarchical architecture combining recurrent neural networks and autoregressive multilayer perceptrons. This hierarchy allows the model to capture temporal patterns at multiple timescales, making it suitable for unconditional audio generation over relatively long spans. Compared with symbolic composition models, SampleRNN illustrates a stronger commitment to end-to-end audio generation, where the model learns directly from raw acoustic sequences rather than from symbolic musical annotations.

The main advantage of raw waveform modeling is that it treats music as sound rather than as notation. It can, at least in principle, learn timbre, performance nuance, noise characteristics, and acoustic texture directly from data. This is a significant conceptual shift: the model no longer needs a separate synthesizer or rendering engine to translate symbolic events into audible music. Instead, the generation target is already an audio signal.

However, raw waveform modeling also introduces severe computational challenges. Because audio signals are sampled at extremely high rates, even a few seconds of sound correspond to tens or hundreds of thousands of sequential prediction steps. Autoregressive waveform models must therefore generate audio sample by sample, making inference slow and making long-range musical structure difficult to control. Later models such as WaveRNN attempted to improve the efficiency of neural audio synthesis, but the basic difficulty of modeling long-form music directly at the waveform level remained significant [53].

For this reason, raw waveform modeling served as a crucial but transitional stage. It demonstrated that neural networks could generate sound itself, but also revealed the difficulty of modeling music simultaneously at the sample level and at the

structural level. These limitations motivated later approaches that use intermediate representations, such as spectrograms or compressed audio latents, to reduce sequence length while preserving essential acoustic information.

3. Spectrogram and Latent Audio Representations

Direct raw waveform modeling demonstrated that neural networks could generate sound itself, but it also revealed the difficulty of working with extremely long sample-level sequences. To reduce this complexity, later approaches began to use intermediate audio representations, such as spectrograms or latent audio embeddings. These representations preserve essential acoustic information while making the generation problem more tractable.

A spectrogram represents sound as a two-dimensional time-frequency structure. Unlike symbolic representations, which encode discrete musical events such as notes or chords, spectrograms capture acoustic properties such as frequency distribution, harmonic content, attack characteristics, and timbral texture. This made it possible to treat audio generation partly as an image-like modeling problem, where convolutional and generative models could learn local time-frequency patterns.

NSynth is an important example of this transition [54]. Engel et al. proposed a WaveNet-style autoencoder trained on a large dataset of individual musical notes. Rather than generating symbolic notes, the model learned temporal embeddings directly from raw audio waveforms and used them to synthesize realistic musical note sounds. More importantly, the learned latent space enabled interpolation between instrumental timbres, allowing the model to generate hybrid sounds between different instruments. This demonstrated that neural music generation could move beyond pitch and rhythm prediction toward the modeling of timbre and sound identity.

Another line of work modeled audio in the spectral domain. GANSynth applied generative adversarial networks to neural audio synthesis by modeling log magnitudes and instantaneous frequencies in a spectrogram-like representation [55]. This approach addressed a key limitation of raw waveform GANs: while GANs allow efficient parallel generation, they often struggle to maintain local waveform coherence. By generating high-resolution spectral representations instead of raw samples directly, GANSynth achieved more coherent audio synthesis and substantially faster generation than autoregressive waveform models.

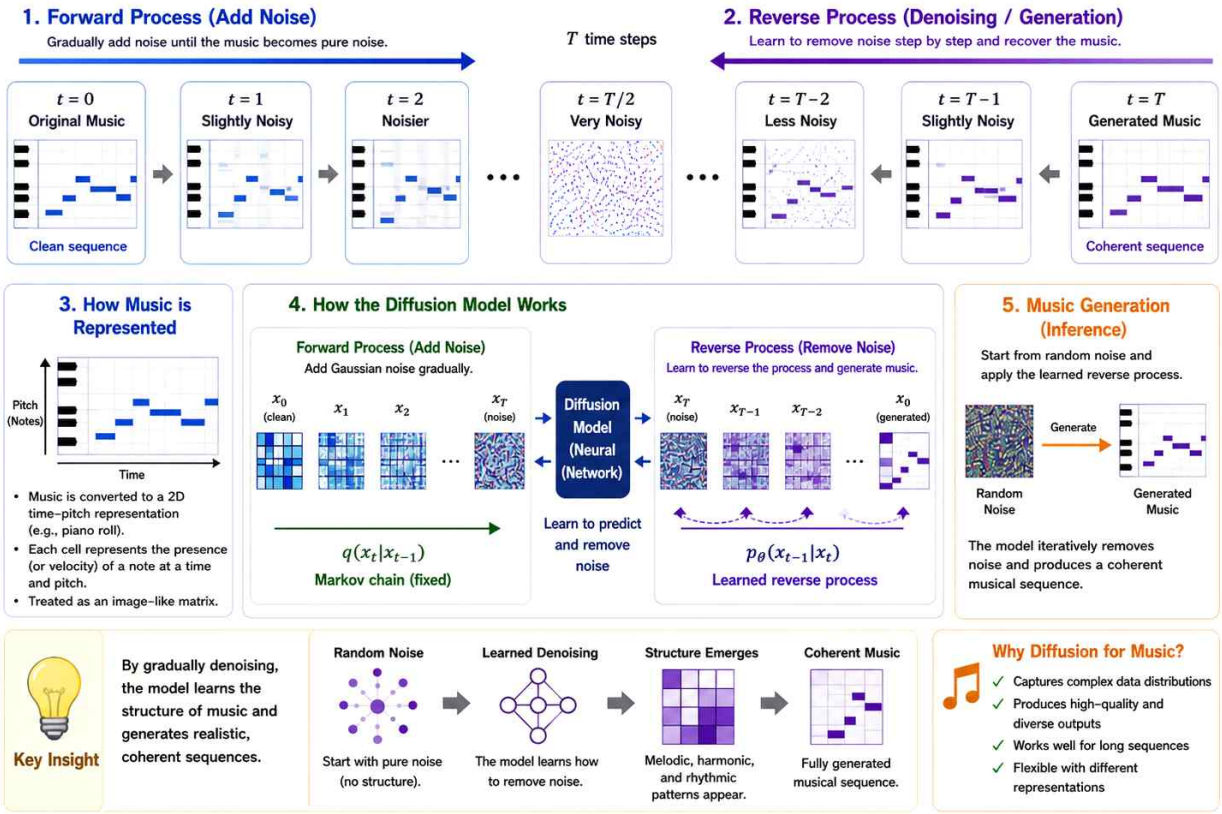
Latent representation learning also became important for audio generation. Vector Quantised Variational Autoencoders (VQ-VAE) introduced a method for learning discrete latent codes from high-dimensional data [56]. Although originally proposed as a general generative modeling framework, this idea later became important in music generation because audio can be compressed into shorter discrete or continuous latent sequences. Once compressed, a separate generative model can learn the distribution of latent codes rather than directly modeling every waveform sample.

These approaches shifted the target of music generation from symbolic event sequences to learned acoustic representations. Spectrograms made it possible to model sound as a time-frequency image, while autoencoder-based methods provided compact latent spaces for timbre, texture, and audio reconstruction. However, these methods also introduced new limitations. Spectrogram-based generation depends on accurate reconstruction of waveform phase or the use of neural vocoders, and latent models may lose fine acoustic detail during compression. As a result, spectrogram and latent audio representations served as an important intermediate stage between raw waveform modeling and later full audio-generation systems.

4. Full Audio Generation

While raw waveform models and neural synthesis systems demonstrated that deep networks could model sound directly, many of them remained limited to short audio fragments, isolated notes, or local timbral synthesis. Full audio generation aimed to extend this direction from sound synthesis to complete musical audio, including longer temporal structure, instrumentation, vocal texture, genre characteristics, and recording-like acoustic detail.

One important precursor to full audio generation was the Sparse Transformer proposed by Child et al. (2019) [57]. Because raw audio sequences are extremely long, applying full self-attention directly to waveform samples is computationally prohibitive. Sparse Transformer addressed this issue by restricting attention patterns, enabling Transformer-based models to handle much longer sequences more efficiently. Although not exclusively a music generation model, it demonstrated that Transformer architectures could be scaled to long raw audio sequences, providing an important technical



[Fig. 5] Diffusion Model for Music Generation: From Noise to Music
Diffusion Models learn to reverse a gradual noising process and transform random noise into coherent musical sequences.

foundation for later full-audio music generation systems.

A representative milestone is OpenAI's Jukebox, proposed by Dhariwal et al. (2020) [58]. Jukebox generates music directly in the raw audio domain, including rudimentary singing, conditioned on genre, artist style, and lyrics. Instead of generating symbolic events such as MIDI notes or chord progressions, the model attempts to generate the final audio signal of a song. This marked an important shift from score-level composition to audio-level music generation, where vocal timbre, instrumental texture, and production-like sound characteristics become part of the generative target.

The key technical challenge in full audio generation is the extremely long context of raw audio. A few minutes of music correspond to millions of waveform samples, making direct autoregressive modeling impractical. Jukebox addressed this problem by using a multi-scale VQ-VAE to compress raw audio into discrete latent codes, and then trained autoregressive Transformer models to generate these compressed codes. The generated latent codes were subsequently decoded back into raw audio. In this sense, Jukebox did not model raw waveform samples directly at the original sampling rate; rather, it modeled compressed audio representations that could later be reconstructed into waveform audio.

This architecture allowed the model to generate more coherent full-song audio than earlier raw waveform models. By conditioning generation on artist, genre, and lyrics, Jukebox also demonstrated a new level of controllability in audio-level music generation. Unlike symbolic systems, which require separate synthesis or rendering stages, Jukebox attempted to generate musical structure and acoustic realization within a unified framework.

However, full audio generation also introduced new limitations. First, the generated audio was difficult to edit or control at a fine musical level. Unlike symbolic representations, where notes, chords, or rhythms can be directly modified, compressed audio latents are less interpretable to human

composers. Second, audio-level models require enormous computational resources because they must learn both long-range musical structure and short-range acoustic detail. Third, although Jukebox could generate multi-minute audio with recognizable style and singing, the resulting music still showed limitations in precise musical planning, lyrical alignment, and high-fidelity production quality.

Thus, full audio generation represented a major conceptual advance, but it also revealed the difficulty of generating music simultaneously as structure, performance, and sound. These limitations motivated later approaches based on diffusion and flow-based generation, which attempted to improve audio fidelity, sampling efficiency, and controllability.

5. Diffusion-based Audio Generation

Diffusion-based models introduced a major shift in audio generation by replacing autoregressive sample-by-sample prediction with iterative denoising. Instead of generating audio sequentially from left to right, a diffusion model begins with random noise and gradually transforms it into structured audio through a learned reverse process. This made diffusion particularly attractive for audio generation, where raw waveform sequences are extremely long and autoregressive generation can be computationally expensive.

One early representative example is DiffWave, proposed by Kong et al. (2020) [59]. DiffWave is a non-autoregressive diffusion probabilistic model for waveform generation. It converts white noise into structured waveforms through a fixed number of denoising steps and can be used for both conditional and unconditional audio generation. In particular, DiffWave demonstrated high-quality waveform synthesis while being significantly faster than autoregressive WaveNet-style models, showing that diffusion could serve as an effective alternative to sample-level autoregressive audio generation.

Diffusion-based audio generation can operate at several representational levels. Some models apply diffusion directly to waveform samples, while others generate spectrograms or compressed audio latent representations. Spectrogram diffusion treats audio as a time-frequency image, generating a mel-spectrogram or related spectral representation that is later converted into waveform audio by a vocoder. Latent diffusion further reduces computational cost by applying the diffusion process in a compressed audio representation space rather than directly in waveform or spectrogram space.

AudioLDM is a representative example of latent diffusion for text-to-audio generation [60]. It learns continuous audio representations in a latent space and conditions generation on text-aligned audio embeddings. By operating in latent space rather than directly on waveform samples, AudioLDM improves computational efficiency while enabling text-guided audio synthesis. Although AudioLDM is a general text-to-audio model rather than a music-only composition system, it is important because it demonstrates how natural language prompts can condition audio-level generative models.

For music-specific generation, Moûsai extended latent diffusion to long-context text-to-music generation [61]. It uses a cascading two-stage latent diffusion architecture to generate high-quality stereo music from text descriptions. Compared with earlier symbolic or waveform-level models, Moûsai illustrates how diffusion can be applied to longer musical audio by operating on compressed representations and by separating coarse and fine generation stages.

The strength of diffusion-based audio generation lies in its ability to model sound as a whole acoustic object rather than as a symbolic event sequence. Diffusion models can capture timbre, spectral texture, noise characteristics, and production-like sound qualities more directly than symbolic models. Moreover, because they can be conditioned on text, audio embeddings, or other control signals, they opened the way for prompt-based music and audio generation.

However, diffusion-based audio generation also has important limitations. First, iterative denoising typically requires multiple sampling steps, which can make generation slower than one-step or few-step models. Second, while diffusion models improve local audio fidelity, long-term musical structure and compositional planning remain difficult. A model may generate convincing sound textures while still failing to maintain coherent form, thematic development, or large-scale harmonic direction. Third, controllability remains limited: text prompts can specify broad style, mood, or instrumentation, but they often struggle to precisely control musical events over time.

Thus, diffusion-based audio generation represented a major advance from symbolic prediction and autoregressive waveform modeling toward holistic audio synthesis. Yet it did not fully solve the problems of long-range musical organization, fine-grained editability, and efficient sampling. These limitations motivated later approaches based on flow

matching, rectified flow, and hybrid architectures that attempt to combine efficient generation with stronger structural control.

6. Flow Matching and Rectified Flow

Although diffusion-based audio generation substantially improved audio quality, its iterative denoising process often requires many sampling steps. This creates a practical limitation for music generation, where long audio duration and high sampling rates already impose a heavy computational burden. Flow matching and rectified flow approaches emerged as attempts to preserve the generative quality of diffusion-like models while improving sampling efficiency.

Flow matching reformulates generation as the learning of a continuous vector field that transports samples from a simple noise distribution to the data distribution. Instead of explicitly learning a step-by-step denoising process, the model learns the velocity field that defines how noise should move toward realistic data. In this sense, flow matching provides a broader continuous-time framework for generative modeling, in which diffusion-like trajectories can be treated as one possible transport path.

MusicFlow is a representative application of flow matching to text-guided music generation [62]. Prajwal et al. proposed a cascaded flow-matching architecture that maps text descriptions to music audio through two stages: one flow-matching network models semantic features, and another models decodable acoustic features. The model also uses a masked prediction objective, allowing it to generalize to related tasks such as music infilling and continuation. Compared with prior text-to-music systems, MusicFlow achieved competitive or superior quality and text coherence while using a smaller model and fewer iterative steps.

Another related approach is MelodyFlow, which applies single-stage flow matching to high-fidelity text-guided music generation and editing [63]. Rather than operating on symbolic tokens or discrete audio codes, MelodyFlow works on continuous latent representations produced by a low-frame-rate 48 kHz stereo variational autoencoder codec. This allows the model to generate and edit high-quality stereo music samples of variable duration using text descriptions. Its emphasis on editing is particularly important because it shifts the goal from merely generating audio to modifying existing musical material in a controllable way.

Rectified flow further develops the same general direction by attempting to make the transport path from noise to data as straight as possible. The motivation is that straighter trajectories can be simulated with fewer numerical integration steps, enabling faster generation. In audio generation, this is especially important because sampling speed strongly affects practical usability. Rectified flow can therefore be understood as part of the broader post-diffusion effort to reduce sampling cost while maintaining audio fidelity.

FlashAudio is a representative rectified-flow-based text-to-audio model [64]. Liu et al. proposed a rectified flow framework for fast and high-fidelity text-to-audio generation, addressing the computational cost of latent diffusion models. By learning straighter flow trajectories and improving sampling efficiency, FlashAudio demonstrated that rectified flow can achieve high-quality audio generation with far fewer sampling steps than conventional diffusion-based methods.

The main advantage of flow matching and rectified flow is efficiency. These methods attempt to retain the expressive power of diffusion-like generative modeling while reducing the number of iterative sampling steps. They also provide a flexible framework for text-conditioned generation, music continuation, infilling, and editing. In this respect, they represent a shift from simply generating complete audio clips toward more interactive and controllable music-generation workflows.

However, these approaches do not by themselves solve the problem of musical structure. Faster sampling improves usability, but it does not guarantee long-term form, thematic development, or compositional intentionality. Like diffusion models, flow-based audio models may generate high-quality local sound textures while still requiring additional mechanisms for global planning and fine-grained musical control. Thus, flow matching and rectified flow should be understood not as a final solution to AI composition, but as an important step toward efficient, controllable audio-level generation.

7. Hybrid and Multimodal Approaches

As audio-level generative models developed, it became increasingly clear that no single architecture fully solved the two central requirements of music generation: global musical coherence and local audio fidelity. Autoregressive models are relatively strong at modeling long-range symbolic or semantic structure, but they often struggle with high-fidelity audio detail.

Diffusion and flow-based models, in contrast, are effective at generating realistic acoustic texture, but they do not inherently guarantee large-scale musical form. This motivated hybrid approaches that combine the structural planning ability of autoregressive models with the acoustic refinement ability of diffusion or flow-based models.

A representative example is SongBloom, proposed by Yang et al. (2025) [65]. SongBloom was designed for coherent full-length song generation and explicitly addressed the difficulty of balancing global coherence with local fidelity. The model uses an interleaved paradigm of autoregressive sketching and diffusion-based refinement: it gradually extends a musical sketch from short to long, while refining acoustic details from coarse to fine. This framework integrates semantic and acoustic context during generation, attempting to overcome the common failure modes of previous language-model and diffusion-based music systems, such as incoherent progression, mismatched lyrics, or locally realistic but structurally weak outputs.

This hybrid direction is important because it reflects a broader recognition that music generation is not a single-level problem. A complete song requires high-level structure, mid-level arrangement, low-level acoustic detail, and alignment between vocal, instrumental, and lyrical elements. Hybrid models attempt to distribute these tasks across complementary generative mechanisms. In this sense, they represent a movement from monolithic music generation toward layered generation, where musical form and sound realization are modeled at different levels and then integrated.

At the same time, music generation has also moved toward multimodal conditioning. Earlier systems were primarily conditioned on symbolic seeds, MIDI events, or text prompts. More recent systems attempt to generate music from other modalities, such as video, dance movement, images, humming, or reference audio. For example, Dance2Music-Diffusion uses a latent diffusion model to generate music from dance videos, demonstrating how visual motion can serve as a conditioning signal for music generation [66]. This direction is particularly relevant because musical creation is often linked to bodily movement, visual imagery, narrative scenes, or emotional context rather than text alone.

A related post-diffusion direction is multimodal rectified flow. OmniFlow extends rectified flow to any-to-any multimodal generation, including text-to-audio and audio-to-image synthesis [67]. Although OmniFlow is not limited to music composition, it demonstrates how rectified flow can be generalized to joint multimodal distributions. Such models suggest a future in which music generation may not be restricted to text-to-audio prompting, but may instead integrate multiple forms of input and output across text, sound, image, video, and gesture.

The main advantage of hybrid and multimodal approaches is that they more closely resemble the multi-layered nature of human musical creation. Human composers do not merely predict the next note or generate sound texture in isolation; they often work from a combination of musical memory, bodily rhythm, imagined timbre, visual imagery, emotional intention, and structural planning. Hybrid and multimodal models attempt to approximate this complexity by combining multiple representational levels and conditioning sources.

However, these approaches also introduce new challenges. First, integrating multiple modules or modalities increases system complexity and makes model behavior harder to interpret. Second, alignment between modalities remains difficult: lyrics, melody, rhythm, motion, and audio texture must be synchronized over time. Third, controllability and editability remain unresolved problems. Even if a model can generate realistic music from rich multimodal input, it may still be difficult for composers to specify precise musical intentions or revise generated results at a fine level. Finally, as models become increasingly multimodal and data-intensive, issues of copyright, dataset transparency, and authorship become more significant.

Thus, hybrid and multimodal approaches represent the current frontier of AI-based music composition. They move beyond the idea of music generation as a single prediction task and instead frame it as the coordinated generation of structure, performance, sound, and multimodal meaning. This development brings AI composition closer to human creative processes, but also makes the problem more complex: the goal is no longer merely to generate plausible music, but to generate music that is structurally coherent, acoustically convincing, controllable, and meaningfully connected to human intention.

VII. Discussion

The technological history reviewed in this paper suggests that AI-based music composition has gradually moved from

local pattern prediction toward increasingly holistic forms of musical representation. This progression can be interpreted not only as a sequence of technical improvements, but also as a gradual approximation of certain structural aspects of human compositional thinking.

Early stochastic models, such as Markov chains and Hidden Markov Models, primarily operated at the level of local pattern recognition. They modeled transitions between notes, chords, or rhythmic states, producing outputs that were statistically plausible within short contexts. In this sense, these models resemble the most localized aspect of composition: choosing the next note, chord, or rhythmic event based on what has immediately preceded it. However, they lacked a broader understanding of musical direction, thematic development, or formal structure.

Recurrent neural networks introduced a more context-sensitive mode of generation. By maintaining hidden states over time, RNNs and LSTMs attempted to preserve musical information across longer sequences. This can be interpreted as a move toward a more human-like consideration of musical context, where each new event is not selected in isolation but shaped by a remembered musical past. Nevertheless, recurrent models still compressed this past into a limited internal state, making it difficult to sustain large-scale formal coherence.

Transformer-based models further changed this relationship to musical memory. By using self-attention, they allowed the model to directly refer to multiple previous events in the sequence. This made it possible to model repetition, variation, motivic recurrence, and long-range dependencies more effectively than recurrent architectures. From the perspective of compositional cognition, this represents a shift from merely remembering the previous flow to comparing and organizing multiple points in musical time. In other words, Transformer-based models began to approximate the structural logic of musical development more explicitly.

The transition toward waveform, spectrogram, and latent audio generation marks an even more significant conceptual shift. Symbolic models generate instructions for music: notes, chords, rhythms, or MIDI events. Audio-level models, by contrast, attempt to generate music as sound itself. This shift is important because human composers do not necessarily begin with fully notated symbolic structures. In many cases, musical ideas first appear as internal auditory images: an imagined sound world that may include melody, harmony, rhythm, instrumentation, timbre, texture, and expressive atmosphere before being translated into notation or performance.

From this viewpoint, audio-level generation can be interpreted as a movement closer to the pre-notational stage of human musical imagination. A composer may internally hear the overall character of a passage, the color of instruments, the density of texture, or the dramatic direction of a section before writing it down. Similarly, modern audio-level models attempt to infer not only symbolic structure but also timbre, spectral texture, vocal quality, and acoustic realization. They do not merely automate the act of writing notes; they increasingly model the formation of a musical sound image.

This does not mean that current AI systems possess intention, consciousness, or artistic agency in the human sense. The analogy should not be overstated. Human composition involves embodied experience, cultural memory, aesthetic judgment, emotion, and intentional communication. Current models remain statistical systems trained on data, and their outputs are shaped by learned distributions rather than subjective purpose. However, the direction of technological development is notable: AI composition systems have evolved from automating peripheral or mechanical aspects of composition toward modeling increasingly central components of the creative process.

In this sense, the history of AI-based music composition can be understood as a gradual shift from the automation of notation-like procedures to the modeling of musical ideation. Earlier systems attempted to determine which musical event should come next. Later systems attempted to preserve context, organize long-range dependencies, and eventually generate audio-level representations of musical sound. This progression suggests that the central question is no longer simply whether AI can compose music, but which layer of the compositional process it is modeling: local decision-making, contextual memory, formal organization, or the emergence of an internal auditory image.

Therefore, the history of AI-based music composition may be read as a movement from the automation of compositional labor toward the simulation of compositional thought. Early systems automated local decisions; modern systems increasingly attempt to generate the auditory image itself. Whether this should be regarded as creativity remains open to debate, but the direction of development suggests that AI

music systems are no longer merely tools for writing music. They are becoming models of how musical ideas can emerge, develop, and become sound.

IIX. Conclusion

This paper reviewed the technological evolution of AI-based music composition, from early rule-based systems and stochastic models to neural sequence models, Transformer architectures, and recent audio-level generative approaches. Across this history, a clear progression can be observed: models have gradually expanded from local event prediction to contextual modeling, long-range dependency modeling, and finally to the generation of audio-level representations.

Early algorithmic systems focused primarily on local musical decisions, such as note-to-note or chord-to-chord transitions. Recurrent neural networks introduced temporal memory, allowing models to preserve musical context over longer sequences. Transformer-based approaches further improved long-context modeling by enabling direct reference to distant musical events through self-attention. More recent waveform, spectrogram, latent audio, diffusion, and flow-based models moved beyond symbolic prediction and began to generate music as sound itself.

This development suggests that AI-based composition has gradually shifted from modeling the external procedures of composition toward modeling more central aspects of musical ideation. Earlier systems automated the selection or arrangement of musical symbols. Later systems increasingly attempt to infer context, structure, timbre, texture, and acoustic realization. In this sense, the evolution of AI composition can be interpreted as a movement from symbolic manipulation toward the generation of holistic musical representations.

However, this does not imply that current AI systems possess human-like creativity or artistic intention. Human composition remains deeply connected to embodied experience, cultural meaning, aesthetic judgment, and communicative purpose. Current generative models still rely on statistical learning from existing data and remain limited in long-term formal planning, fine-grained controllability, authorship, and interpretability.

Nevertheless, the trajectory is significant. The central question is no longer simply whether AI can generate plausible music, but which layer of the compositional process it is beginning to model. As AI systems move from local pattern prediction toward audio-level and multimodal generation, they increasingly approximate the structural pathway through which musical ideas are formed, organized, and externalized as sound. Future research should therefore focus not only on improving generation quality, but also on developing models that better support musical intention, structural control, editability, and meaningful human–AI collaboration.

AI Assistance Notice

The figures on this papers are generated with AI.

References

[1] HILLER, L. "Musical Composition with a High-Speed Digital Computer." Machines Models of Music (1958): 9-21.

[2] Gill, Stanley. "A Technique for the Composition of Music in a Computer." The Computer Journal 6.2 (1963): 129-133.

[3] Rothgeb, John Edward. “Harmonizing the unfigured bass: A computational study. Yale University, 1968.

[4] Thomas, Marilyn Taft. Vivace: A Rule Based Al System for Composition. Ann Arbor, MI: Michigan Publishing, University of Michigan Library, 1985.

[5] Löthe, Mathis. "Knowledge based automatic composition and variation of melodies for minuets in early classical style." Annual Conference on Artificial Intelligence. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999.

[6] Fernández, Jose D., and Francisco Vico. "AI methods in algorithmic composition: A comprehensive survey." Journal of Artificial Intelligence Research 48 (2013): 513-582.

[7] Ebcioğlu, Kemal. Computer counterpoint. Ann Arbor, MI: Michigan Publishing, University of Michigan Library, 1980.

[8] Shottstaedt, William. "Automatic counterpoint." Current directions in computer music research. 1989. 199-214.

[9] Horner, Andrew, and David E. Goldberg. "Genetic algorithms and computer-assisted music composition." Vol. 51. Ann Arbor, MI: Michigan Publishing, University of Michigan Library, 1991.

[10] Biles, John. "GenJam: A genetic algorithm for generating jazz solos." ICMC. Vol. 94. 1994.

[11] McIntyre, Ryan A. "Bach in a box: The evolution of four part baroque harmony using the genetic algorithm." Proceedings of the first ieee conference on evolutionary computation. ieee world congress on computational intelligence. IEEE, 1994.

[12] Ariza, Christopher. "Two pioneering projects from the early history of computer-aided algorithmic composition." Computer Music Journal 35.3 (2011): 40-56.

[13] Ames, Charles. "Automated composition in retrospect: 1956-1986." Leonardo (1987): 169-185.

[14] Pinkerton, Richard C. "Information theory and melody." Scientific American 194.2 (1956): 77-87.

[15] Tipei, Sever. MP1: a computer program for music composition. Ann Arbor, MI: Michigan Publishing, University of Michigan Library, 1975.

[16] Ames, Charles, and Michael Domino. "Cybernetic Composer: an overview." Understanding Music with AI: Perspectives on

Music Cognition (1992): 186-205.
[17] Ponsford, Dan, Geraint Wiggins, and Chris Mellish. "Statistical learning of harmonic movement." Journal of New Music Research 28.2 (1999): 150-177.
[18] Grachten, Maarten. "JIG: Jazz improvisation generator." Proceedings of the MOSART Workshop on Current Research Directions in Computer Music. Barcelona, Spain: Pompeu Fabra University Publishers, 2001.
[19] Thom, Belinda. "BoB: an interactive improvisational music companion." Proceedings of the fourth international conference on Autonomous agents. 2000.
[20] Biyikoglu, Kaan M. "A Markov model for chorale harmonization." Preceedings of the 5 th Triennial ESCOM Conference. 2003.
[21] Allan, Moray. "Harmonising chorales in the style of Johann Sebastian Bach." Master's Thesis, School of Informatics, University of Edinburgh (2002).
[22] Morris, Dan, Ian Simon, and Sumit Basu. "Exposing parameters of a trained dynamic model for interactive music creation." AAAI'08 Proceedings of the 23rd national conference on Artificial intelligence-Volume 2. 2008.
[23] Farbood, Mary, and Bernd Schöner. "Analysis and synthesis of Palestrina-style counterpoint using Markov chains." ICMC. 2001
[24] Conklin, Darrell, and Ian H. Witten. "Multiple viewpoint systems for music prediction." Journal of New Music Research 24.1 (1995): 51-73.
[25] Pachet, François. "Interacting with a musical learning system: The continuator." International conference on music and artificial intelligence. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002.
[26] Schulze, Walter. A formal language theory approach to music generation. Diss. Stellenbosch: University of Stellenbosch, 2010.
[27] Pasquier, Philippe. "Realtime generation of harmonic progressions using controlled Markov selection." Proceedings of ICCC-X-computational creativity conference. 2010.
[28] Martin, Aengus, Craig T. Jin, and Oliver Bown. "Implementation of a real-time musical decision-maker." Proceedings of the Australasian Computer Music Conference. 2012.
[29] Dubnov, Shlomo, et al. "Using machine-learning methods for musical style modeling." Computer 36.10 (2003): 73-80.
[30] Assayag, Gérard, and Shlomo Dubnov. "Using factor oracles for machine improvisation." Soft Computing 8.9 (2004)
[31] Shibata, N. "A neural network-based method for chord/note scale association with melodies." NEC research & development 32.3 (1991): 453-459.
[32] Verbeurgt, K., Fayer, M., & Dinolfo, M. (2004). A hybrid Neural-Markov approach for learning to compose music by example. In Proceedings of the Canadian Conference on Advances in Artificial Intelligence, pp. 480－484
[33] Hild, Hermann, Johannes Feulner, and Wolfram Menzel. "HARMONET: A neural net for harmonizing chorales in the style of JS Bach." Advances in neural information processing systems 4 (1991).
[34] Ban, B. H. (2017). Method for composing music using deep reinforcement learning. Korean Patent Application No. 10-2017-0024223, filed February 23, 2017; published as KR10-2017-0128073A, November 22, 2017.
[35] Todd, Peter M. "A connectionist approach to algorithmic composition." Computer Music Journal 13.4 (1989): 27-43.
[36] Franklin, Judy A. "Multi-phase learning for jazz improvisation and interaction." Proceedings of the Eight Biennial Symposium on Art and Technology. 2001.
[37] B. H. Ban, “Method for composing music using deep reinforcement learning,” Korean Patent Application No. 10-2017-0024223, filed Feb. 23, 2017, published as KR10-2017-0128073A, Nov. 22, 2017.
[38] B. H. Ban, J. H. Park, and Y. E. Jung, “Method for composing music based on recurrent neural networks,” Korean Patent Application No. 10-2017-0022493, filed Feb. 20, 2017, published as KR10-2017-0128070A, Nov. 22, 2017.
[39] Eck, Douglas, and Jürgen Schmidhuber. "Learning the long-term structure of the blues." International Conference on Artificial Neural Networks. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002.
[40] Sturm, Bob L., et al. "Music transcription modelling and composition using deep learning." arXiv preprint arXiv:1604.08723 (2016).
[41] Colombo, Florian, et al. "Algorithmic composition of melodies with deep recurrent neural networks." arXiv preprint arXiv:1606.07251 (2016).
[42] Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." arXiv preprint arXiv:1409.0473 (2014).
[43] Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).
[44] Oore, Sageev, et al. "Learning to create piano performances." NIPS 2017 Workshop on Machine Learning for Creativity and Design. 2017.
[45] Goolge Magenta, https://magenta.tensorflow.org
[46] Keerti, Gullapalli, et al. "Attentional networks for music generation." Multimedia Tools and Applications 81.4 (2022)
[47] Huang, Cheng-Zhi Anna, et al. "Music transformer." arXiv preprint arXiv:1809.04281 (2018).
[48] Payne, C. (2019). MuseNet. OpenAI.
[49] Huang, Yu-Siang, and Yi-Hsuan Yang. "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions." Proceedings of the 28th ACM

international conference on multimedia. 2020.

[50] Yu, Botao, et al. "Museformer: Transformer with fine-and coarse-grained attention for music generation." Advances in neural information processing systems 35 (2022): 1376-1388.

[51] Oord, Aaron van den, et al. "Wavenet: A generative model for raw audio." arXiv preprint arXiv:1609.03499 (2016).

[52] Mehri, Soroush, et al. "SampleRNN: An unconditional end-to-end neural audio generation model." arXiv preprint arXiv:1612.07837 (2016).

[53] Kalchbrenner, Nal, et al. "Efficient neural audio synthesis." International Conference on Machine Learning. PMLR, 2018.

[54] Engel, Jesse, et al. "Neural audio synthesis of musical notes with wavenet autoencoders." International conference on machine learning. PMLR, 2017.

[55] Engel, Jesse, et al. "Gansynth: Adversarial neural audio synthesis." arXiv preprint arXiv:1902.08710 (2019).

[56] Van Den Oord, Aaron, and Oriol Vinyals. "Neural discrete representation learning." Advances in neural information processing systems 30 (2017).

[57] Child, Rewon, et al. "Generating long sequences with sparse transformers." arXiv preprint arXiv:1904.10509 (2019).

[58] Dhariwal, Prafulla, et al. "Jukebox: A generative model for music." arXiv preprint arXiv:2005.00341 (2020).

[59] [60] H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. P. Mandic, W. Wang, and M. D. Plumbley, "AudioLDM: Text-to-audio generation with latent diffusion models," in Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 21450 − 21474, 2023.

[60] Liu, Haohe, et al. "Audioldm: Text-to-audio generation with latent diffusion models." arXiv:2301.12503 (2023).

[61] Schneider, Flavio, et al. "Mo\^ usai: Text-to-music generation with long-context latent diffusion." arXiv preprint arXiv:2301.11757 (2023).

[62] Prajwal, K. R., et al. "MusicFlow: Cascaded flow matching for text guided music generation." arXiv preprint arXiv:2410.20478 (2024).

[63] Lan, Gael Le, et al. "High fidelity text-guided music editing via single-stage flow matching." arXiv preprint arXiv:2407.03648 (2024).

[64] Liu, Huadai, et al. "Flashaudio: Rectified flow for fast and high-fidelity text-to-audio generation." Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025.

[65] Yang, Chenyu, et al. "Songbloom: Coherent song generation via interleaved autoregressive sketching and diffusion refinement." arXiv preprint arXiv:2506.07634 (2025).

[66] Zhang, Chaoyang, and Yan Hua. "Dance2Music-Diffusion: leveraging latent diffusion models for music generation from dance videos." EURASIP Journal on Audio, Speech, and Music Processing 2024.1 (2024): 48.

[67] Li, Shufan, et al. "Omniflow: Any-to-any generation with multi-modal rectified flows." Proceedings of the Computer Vision and Pattern Recognition Conference. 2025.